

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**
Образовательная программа **Мобильные и сетевые технологии**
Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине «Проектирование и реализация баз данных»

Тема: Введение в СУБД MongoDB. Установка MongoDB. Начало работы с БД. Работа с БД в СУБД MongoDB.

Выполнил	Колпаков А.С.; К 3239.
Проверил	Говорова М.М.
Дата	28.05.2026

Санкт-Петербург 2026

1 Цель работы

- 1) Овладеть практическими навыками установки СУБД MongoDB.
- 2) Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

2 Практические задания

Практическое задание для ЛР 6.1:

- Установите MongoDB для обеих типов систем (32/64 бита).
- Проверьте работоспособность системы запуском клиента mongo.
- Выполните методы:
 - a) db.help()
 - b) db.help
 - c) db.stats()
- Создайте БД learn.
- Получите список доступных БД.
- Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.
- Просмотрите список текущих коллекций.
- Переименуйте коллекцию unicorns.
- Просмотрите статистику коллекции.
- Удалите коллекцию.
- Удалите БД learn.

Практическое задание для ЛР 6.2: см. пункт 5 “Выполнение лабораторной работы №6.2”

4 Выполнение лабораторной работы №6.1

4.1 Установка СУБД MongoDB

MongoDB была установлена на macOS с помощью пакетного менеджера Homebrew. Так как современная macOS является 64-битной системой, была установлена актуальная версия MongoDB Community Edition для 64-битной архитектуры.

Установка выполнялась командой:

```
brew install mongodb-community
```

```
> brew install mongodb-community
Inspect the formula dependency plan before installing with `brew install --ask`.
Enable ask mode by setting `HOMEBREW_ASK=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
-- Fetching downloads for: mongodb-community
Bottle fat (12.1.0)
Bottle Manifest ada-url (3.4.4)
Bottle ada-url (3.4.4)
Bottle brotli (1.2.0)
Bottle Manifest c-ares (1.34.6)
Bottle c-ares (1.34.6)
Bottle Manifest hdrhistogram_c (0.11.9)
Bottle hdrhistogram_c (0.11.9)
Bottle Manifest libnghttp2 (1.69.0)
Bottle libnghttp2 (1.69.0)
Bottle Manifest libnghttp3 (1.15.0)
Bottle libnghttp3 (1.15.0)
Bottle Manifest ca-certificates (2026-05-14)
Bottle Manifest openssl@3 (3.6.2)
Bottle Manifest libngtcp2 (1.22.1)
Bottle ca-certificates (2026-05-14)
Bottle Manifest brotli (1.2.0)
Bottle Manifest lhttp (9.4.1)
Downloaded 291.8KB/291.8KB
Downloaded 8.7KB/ 8.7KB
Downloaded 349.8KB/349.8KB
Downloaded 794.0KB/794.0KB
Downloaded 7.5KB/ 7.5KB
Downloaded 304.4KB/304.4KB
Downloaded 7.8KB/ 7.8KB
Downloaded 43.8KB/ 43.8KB
Downloaded 7.3KB/ 7.3KB
Downloaded 232.6KB/232.6KB
Downloaded 7.3KB/ 7.3KB
Downloaded 189.8KB/189.8KB
Downloaded 1.7KB/ 1.7KB
Downloaded 12.0KB/ 12.0KB
Downloaded 9.7KB/ 9.7KB
Downloaded 112.1KB/112.1KB
Downloaded 8.0KB/ 8.0KB
Downloaded 7.2KB/ 7.2KB
```

Рис. 1: Установка утилиты

После установки запускаем сервер MongoDB:

```
brew services start mongodb-community
```

Работоспособность MongoDB была проверена запуском клиента mongosh. Подключение к локальному серверу MongoDB прошло успешно.

Для проверки работоспособности была использована программа:

```
mongosh
```

После выполнения команды была открыта оболочка MongoDB Shell (mongosh)

```
> mongosh
Current Mongosh Log ID: 6a185d2bbc2655abca876114
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.2.9
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-05-28T18:19:15.868+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----
```

Рис. 2: Подключение к MongoDB

4.2 Вызов базовых методов и создание БД

Для просмотра списка доступных методов объекта базы данных использовалась команда:

```
db.help()
```

Команда отображает основные функции и методы для работы с MongoDB.

```
test> db.help()

Database Class:

getMongo           Returns the current database connection
getName           Returns the name of the DB
getCollectionNames Returns an array containing the names of all collections in the current database.
getCollectionInfos Returns an array of documents with collection information, i.e. collection name and options, for the current database.
runCommand         Runs an arbitrary command on the database.
adminCommand       Runs an arbitrary command against the admin database.
aggregate          Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
getSiblingDB       Returns another database without modifying the db variable in the shell environment.
getCollection      Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
dropDatabase       Removes the current database, deleting the associated data files.
createUser         Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already exists on the database.
updateUser         Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
logout            Ends the current authentication session. This function has no effect if the current session is not authenticated.
dropUser          Removes the user from the current database.
dropAllUsers       Removes all users from the current database.
auth              Allows a user to authenticate to the database from within the shell.
grantRolesToUser   Grants additional roles to a user.
revokeRolesFromUser Removes a one or more roles from a user on the current database.
getUser           Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which the method runs.
```

Рис. 3: Вызов метода db.help()

```
test> db.stats()
|
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
```

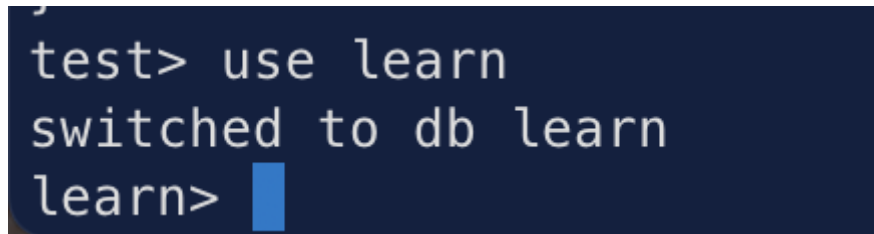
Рис. 4: Вызов метода db.stats()

Эта команда выводит статистику текущей базы данных: имя БД, количество коллекций, объектов, размер данных и другую служебную информацию. В методичке отдельно указано, что db.stats() показывает текущее состояние базы данных и занимаемый ей размер.

Для создания базы данных использовалась команда:

```
use learn
```

Командой `use learn` была выбрана база данных `learn`. Фактически база будет создана после вставки первого документа.



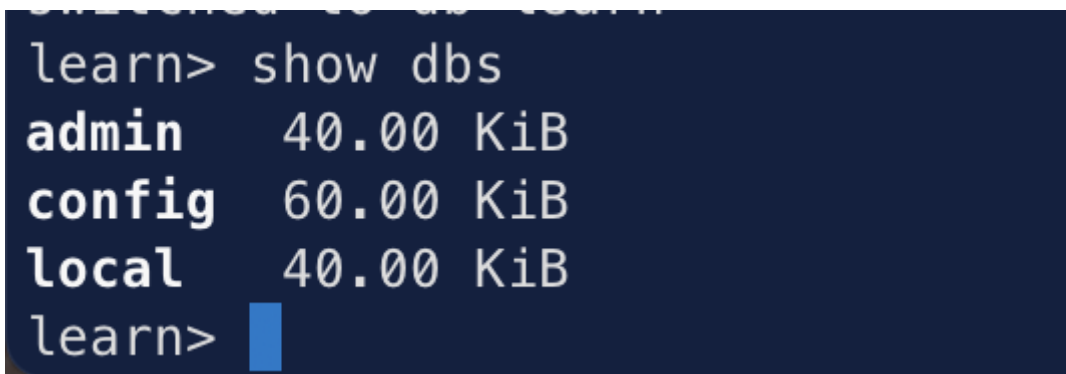
```
test> use learn
switched to db learn
learn>
```

Рис. 5: Переключение контекста на новую БД

Для просмотра списка всех существующих БД была использована команда:

```
show dbs
```

В результате выполнения команды выводятся имена баз данных и объем занимаемого ими дискового пространства.



```
learn> show dbs
admin    40.00 KiB
config   60.00 KiB
local    40.00 KiB
learn>
```

Рис. 6: Результат выполнения команды `show dbs`

В списке отображаются учебные и системные базы данных MongoDB. Системные базы данных:

admin — Системная база данных MongoDB, предназначенная для хранения информации о пользователях, ролях и административных настройках сервера.

local — Системная база данных, используемая MongoDB для хранения служебной информации replica set и внутренних данных сервера.

База данных **learn** ещё не отображалась в списке, поскольку MongoDB создает базы данных физически только после добавления первого документа или коллекции.

4.3 Работа с коллекциями и документами

Для создания коллекции unicorns была использована такая команда:

```
db.unicorns.insertOne({name: 'Aurora', gender: 'f',  
weight: 450})
```

```
learn> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a18604ebc2655abca876115')  
}  
learn> █
```

Рис. 7: Создание коллекции unicorns

Для просмотра коллекций в БД была использована команда:

```
show collections
```

```
learn> show collections  
unicorns  
learn> █
```

Рис. 8: Получение списка коллекций

Для переименования коллекции была использована команда:

```
db.unicorns.renameCollection("horses")
```

```
unicorns  
learn> db.unicorns.renameCollection("horses")  
{ ok: 1 }  
learn> █
```

Рис. 9: Успешное переименование коллекции

Для получения статистики коллекции была использована команда:

```
db.horses.stats()
```

```
learn> db.horses.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata={formatVersion:1},assert={commit_timestamp=none,durable_timestamp=none,read_timestamp=none,write_timestamp=none},block_allocation=best,block_compressor=snappy,cache_resident=false,checksumon=colgroups,collator=,columns=,dictionary=0,encryption={keyid=,names=},exclusive=false,extractors=,format=btree,huffman_keys=,huffman_values=,ignore_in_memory_cache_size=false,immutable=false,import={compare_timestamp=oldest_timestamp,enabled=false,file_metadata=,metadata_file=,panic_corrupt=true,repair=false},internal_item_max=0,internal_key_max=0,internal_key_truncate=true,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64MB,logs={enabled=true},lsm={auto_throttle=,bloom=,bloom_bit_count=,bloom_config=,bloom_hash_count=,bloom_oldest=,chunk_count_limit=,chunk_max=,chunk_size=,merge_custom={prefix=,start_generation=,suffix=},merge_max=,merge_min=},memory_page_image_max=0,memory_page_max=10m,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_pct=90,tiered_storage={auth_token=,bucket=,bucket_prefix=,cache_direct_arry=,local_retention=300,names=,object_target_size=0},type=file,value_format=u,verbose=1,write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:tablescollection-a95725fa-bd44-423e-bdfb-dbc5f6be7c1c',
    version: 'WiredTiger 12.0.0: (November 15, 2024)',
    autocommmit: {
      'retries for readonly operations': 0,
      'retries for update operations': 0
    },
    backup: {
      'total modified incremental blocks with compressed data': 0,
      'total modified incremental blocks without compressed data': 0
    },
    'block-disagg': {
      'Bytes read from the shared history store in SLS': 0,
      'Bytes written to the shared history store in SLS': 0,
      'Disaggregated block manager ops': 0
    }
  }
}
```

Рис. 10: Успешное получение статистики коллекции

Для удаления коллекции была использована команда:

```
db.horses.drop()
```

```
learn> db.horses.drop()
true
learn>
```

Рис. 11: Успешное удаление коллекции

Для удаления БД была использована команда:

```
db.dropDatabase()
```

```
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn>
```

Рис. 12: Успешное удаление БД

5 Выполнение лабораторной работы №6.2

5.1 Вставка документов в коллекцию

Выполнение практического задания 2.1.1

С помощью команды use была создана БД learn. После этого, используя первый способ для добавления документов в коллекцию из описания ЛР 6.2, в коллекцию unicorns были добавлены документы:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'],
weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'],
weight: 450, gender: 'f', vampires: 43});
```

```

db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'],
weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooodoodles', loves: ['apple'], weight:
575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot',
'chocolate'], weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'],
weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight:
690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'],
weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'],
weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'],
weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'],
weight: 540, gender: 'f'});

```

```

learn> db.unicorns.insertOne({
|   name: 'Horny',
|   loves: ['carrot', 'papaya'],
|   weight: 600,
|   gender: 'm',
|   vampires: 63
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a186352bc2655abca876116')
}
learn>

```

Рис. 13: Добавление документа в коллекцию первым способом

И также используя второй способ из описания, был добавлен еще один документ:

```

learn> document = {
|   name: 'Dunx',
|   loves: ['grape', 'watermelon'],
|   weight: 704,
|   gender: 'm',
|   vampires: 165
| }
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a1863ccbc2655abca876121')
}
learn>

```

Рис. 14: Добавление документа в коллекцию вторым способом

После этого была сделана проверка содержимого коллекции с помощью метода `find()`:

```

learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a186352bc2655abca876116'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a186395bc2655abca876117'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a18639abc2655abca876118'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a1863a0bc2655abca876119'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',

```

Рис. 15: Проверка содержимого коллекции с помощью метода `find`

5.2 Выборка данных из БД

Выполнение практического задания 2.2.1

В ходе выполнения задания были сформированы запросы выборки документов из коллекции `unicorns` с использованием метода `find()`.

1) Получение списка самцов:

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a186352bc2655abca876116'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a186395bc2655abca876117'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a18639abc2655abca876118'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a1863a0bc2655abca876119'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a1863a4bc2655abca87611a'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a1863a8bc2655abca87611b'),
    name: 'Ayna',

```

Рис. 16: Список всех самцов с сортировкой по имени

Запрос выполняет выборку всех самцов единорогов и сортирует результат по имени в алфавитном порядке.

2) Получение первых трех самок:

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a186395bc2655abca876117'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1863a8bc2655abca87611b'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a1863b9bc2655abca87611e'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> 
```

Рис. 17: Список первых трех самок с сортировкой по имени

Запрос выводит только первые три документа после сортировки списка самок по имени.

3) Получение всех самок, любящих carrot:

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a186395bc2655abca876117'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1863a4bc2655abca87611a'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a1863c2bc2655abca876120'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Рис. 18: Список всех самок, любящих carrot

4) Получение только первой такой самки через findOne:

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a186395bc2655abca876117'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn>
```

Рис. 19: Первая самка, любящая carrot, через findOne

5) Получение только первой такой самки через limit:

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a186395bc2655abca876117'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>
```

Рис. 20: Первая самка, любящая carrot, через limit

Выполнение практического задания 2.2.2

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях.

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {loves: 0, _id: 0}
| ).sort({name: 1})
|
| [
|   { name: 'Dunx', weight: 704, gender: 'm', vampires: 165 },
|   { name: 'Horny', weight: 600, gender: 'm', vampires: 63 },
|   { name: 'Kenny', weight: 690, gender: 'm', vampires: 39 },
|   { name: 'Pilot', weight: 650, gender: 'm', vampires: 54 },
|   { name: 'Raleigh', weight: 421, gender: 'm', vampires: 2 },
|   { name: 'Roooooodles', weight: 575, gender: 'm', vampires: 99 },
|   { name: 'Unicrom', weight: 984, gender: 'm', vampires: 182 }
| ]
learn> 
```

Рис. 21: Список самцов, исключая информацию о предпочтениях

Выполнение практического задания 2.2.3

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a1863ccbc2655abca876121'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1863c2bc2655abca876120'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a1863bdbbc2655abca87611f'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a1863b9bc2655abca87611e'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a1863b4bc2655abca87611d'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Рис. 22: Список единорогов в обратном порядке добавления

Выполнение практического задания 2.2.4

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find(
|   {},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| )
|
| [
|   { name: 'Horny', loves: [ 'carrot' ] },
|   { name: 'Aurora', loves: [ 'carrot' ] },
|   { name: 'Unicrom', loves: [ 'energon' ] },
|   { name: 'Rooooooodles', loves: [ 'apple' ] },
|   { name: 'Solnara', loves: [ 'apple' ] },
|   { name: 'Ayna', loves: [ 'strawberry' ] },
|   { name: 'Kenny', loves: [ 'grape' ] },
|   { name: 'Raleigh', loves: [ 'apple' ] },
|   { name: 'Leia', loves: [ 'apple' ] },
|   { name: 'Pilot', loves: [ 'apple' ] },
|   { name: 'Nimue', loves: [ 'grape' ] },
|   { name: 'Dunx', loves: [ 'grape' ] }
| ]
learn>
```

Рис. 23: Список единорогов с названием первого предпочтения, исключая поле `_id`

В ходе выполнения задания была выполнена выборка документов из коллекции `unicorns` с использованием проекции полей.

Для массива `loves` был применен оператор `$slice`, позволяющий вывести только первый элемент массива предпочтений. Также из результата был исключен идентификатор `_id`.

5.3 Логические операторы

Выполнение практического задания 2.3.1

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```

learn> db.unicorns.find(
|   {
|     gender: 'f',
|     weight: {$gte: 500, $lte: 700}
|   },
|   {
|     _id: 0
|   }
| )
|
| [
|   {
|     name: 'Solnara',
|     loves: [ 'apple', 'carrot', 'chocolate' ],
|     weight: 550,
|     gender: 'f',
|     vampires: 80
|   },
|   {
|     name: 'Leia',
|     loves: [ 'apple', 'watermelon' ],
|     weight: 601,
|     gender: 'f',
|     vampires: 33
|   },
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
learn>

```

Рис. 24: Список единорогов весом от 500 кг до 700 кг, исключая поле `_id`

В ходе выполнения задания были использованы логические операторы сравнения `$gte` и `$lte`.

Выполнение практического задания 2.3.2

Вывести список самцов единорогов весом от полутонны и предпочитающих `grape` и `lemon`, исключив вывод идентификатора.

```

]
learn> db.unicorns.find(
|   {
|     gender: 'm',
|     weight: {$gte: 500},
|     loves: {$all: ['grape', 'lemon']}
|   },
|   {
|     _id: 0
|   }
| )
|
| [
|   {
|     name: 'Kenny',
|     loves: [ 'grape', 'lemon' ],
|     weight: 690,
|     gender: 'm',
|     vampires: 39
|   }
| ]
learn>

```

Рис. 25: Список самцов весом от 500 кг и предпочитающих grape и lemon, исключая поле `_id`

В ходе выполнения задания был использован оператор `$all`, предназначенный для поиска документов, массив которых содержит все указанные элементы.

Выполнение практического задания 2.3.3

Найти всех единорогов, не имеющих ключ `vampires`.

```

]
learn> db.unicorns.find(
|   {
|     vampires: {$exists: false}
|   },
|   {
|     _id: 0
|   }
| )
|
| [
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
learn>

```

Рис. 26: Список единорогов, не имеющих поле `vampires`

В ходе выполнения задания был использован оператор `$exists`, позволяющий проверить наличие или отсутствие поля в документе.

Выполнение практического задания 2.3.4

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find(
|   {
|     gender: 'm'
|   },
|   {
|     _id: 0,
|     name: 1,
|     loves: {$slice: 1}
|   }
| ).sort({name: 1})
|
| [
|   { name: 'Dunx', loves: [ 'grape' ] },
|   { name: 'Horny', loves: [ 'carrot' ] },
|   { name: 'Kenny', loves: [ 'grape' ] },
|   { name: 'Pilot', loves: [ 'apple' ] },
|   { name: 'Raleigh', loves: [ 'apple' ] },
|   { name: 'Rooooooodles', loves: [ 'apple' ] },
|   { name: 'Unicrom', loves: [ 'energon' ] }
| ]
learn>
```

Рис. 27: Список самцов с информацией об их первом предпочтении, сортировка по имени

5.4 Запросы к вложенным объектам

Выполнение практического задания 3.1.1

1) В ходе выполнения задания была создана коллекция **towns**, содержащая вложенные документы **mayor**. Были добавлены сведения о городах, населении, известных особенностях и мэрах.

```

learn> db.towns.insertOne({
|   name: "Punxsutawney",
|   populatiuon: 6200,
|   last_sensus: ISODate("2008-01-31"),
|   famous_for: [""],
|   mayor: {
|     name: "Jim Wehrle"
|   }
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a18697bbc2655abca876122')
}
learn> db.towns.insertOne({
|   name: "New York",
|   populatiuon: 22200000,
|   last_sensus: ISODate("2009-07-31"),
|   famous_for: ["status of liberty", "food"],
|   mayor: {
|     name: "Michael Bloomberg",
|     party: "I"
|   }
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a186983bc2655abca876123')
}
learn> db.towns.insertOne({
|   name: "Portland",
|   populatiuon: 528000,
|   last_sensus: ISODate("2009-07-20"),
|   famous_for: ["beer", "food"],
|   mayor: {
|     name: "Sam Adams",
|     party: "D"
|   }
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a186987bc2655abca876124')
}

```

Рис. 28: Создание коллекции towns и добавление документов

2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```

learn> db.towns.find(
|   {"mayor.party": "I"},
|   {_id: 0, name: 1, mayor: 1}
| )
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn>

```

Рис. 29: Список городов с независимыми мэрами

В ходе выполнения задания был сформирован запрос к вложенному объекту с использованием оператора точки. Были выбраны города, мэры которых являются независимыми (`party = "I"`).

3) Сформировать запрос, который возвращает список беспартийных мэров (`party` отсутствует) . Вывести только название города и информацию о мэре.

```
learn> db.towns.find(
|   {"mayor.party": {$exists: false}},
|   {_id: 0, name: 1, mayor: 1}
| )
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn>
```

Рис. 30: Список городов с беспартийными мэрами

В ходе выполнения задания был использован оператор `$exists` для поиска документов, в которых отсутствует поле `party` у вложенного объекта `mayor`.

Выполнение практического задания 3.1.2

- 1) *Сформировать функцию для вывода списка самцов единорогов.*
- 2) *Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.*
- 3) *Вывести результат, используя forEach*

Функция для вывода самцов:

```

learn> function maleUnicorns() {
|   return db.unicorns.find({gender: 'm'})
| }
[Function: maleUnicorns]
learn> maleUnicorns()
[
  {
    _id: ObjectId('6a186352bc2655abca876116'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a18639abc2655abca876118'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a1863a0bc2655abca876119'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a1863aebc2655abca87611c'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a1863b4bc2655abca87611d'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,

```

Рис. 31: Создание и вызов функции для вывода списка единорогов самцов

В ходе выполнения задания была создана JavaScript-функция для получения списка самцов единорогов.

Создание курсора:

```
learn> var cursor = maleUnicorns().sort({name: 1}).limit(2)
learn> 
```

Рис. 32: Создание курсора на основе функции

Вывод через forEach:

```
learn> cursor.forEach(function(obj) {
|   print(obj.name)
| })
Dunx
Horny
learn> 
```

Рис. 33: Вывод результата с помощью метода forEach

На основе функции был создан курсор с сортировкой по имени и ограничением количества документов. Для вывода результатов использовался метод `forEach()`.

5.5 Агрегированные запросы

Выполнение практического задания 3.2.1

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({
|   gender: 'f',
|   weight: {$gte: 500, $lte: 600}
| }).count()
(node:15364) [MONGODB DRIVER] Warning: cursor.count is deprecated and will be removed in the next major version, please use 'collection.estimatedDocumentCount' or 'collection.countDocuments' instead
(Use 'node --trace-warnings ...' to show where the warning was created)
2
learn> 
```

Рис. 34: Вывод количества самок единорогов весом от 500 кг и до 600 кг

Выполнение практического задания 3.2.2

Вывести список уникальных предпочтений единорогов.

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> 
```

Рис. 35: Вывод списка уникальных предпочтений единорогов

В ходе выполнения задания был использован метод `distinct()` для получения уникальных значений предпочтений единорогов.

Выполнение практического задания 3.2.3

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([
|   {
|     $group: {
|       _id: "$gender",
|       count: {$sum: 1}
|     }
|   }
| ])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn>
```

Рис. 36: Вывод количества самцов и самок

В ходе выполнения задания был использован агрегатный запрос `aggregate()` с оператором `$group` для подсчета количества документов по полю `gender`.

5.6 Редактирование данных

Выполнение практического задания 3.3.1

1. *Выполнить команду:*
`> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})`
2. *Проверить содержимое коллекции `unicorns`.*

```
learn> db.unicorns.insertOne({
|   name: 'Barney',
|   loves: ['grape'],
|   weight: 340,
|   gender: 'm'
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a186be0bc2655abca876125')
}
learn>
```

Выполнение практического задания 3.3.2

1. *Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.*
2. *Проверить содержимое коллекции `unicorns`.*

```

learn> db.unicorns.updateOne(
|   {name: 'Ayna'},
|   {$set: {weight: 800, vampires: 51}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

Рис. 37: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$set` для изменения отдельных полей документа.

Выполнение практического задания 3.3.3

1. Для самца единорога *Raleigh* внести изменения в БД: теперь он любит *рэдбул*.
2. Проверить содержимое коллекции *unicorns*.

```

learn> db.unicorns.updateOne(
|   {name: 'Raleigh'},
|   {$set: {loves: ['redbull']}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a1863b4bc2655abca87611d'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn>

```

Рис. 38: Изменение документа и проверка содержимого

Выполнение практического задания 3.3.4

1. *Всем самцам единорогов увеличить количество убитых вампиров на 5.*
2. *Проверить содержимое коллекции `unicorns`.*

```
learn> db.unicorns.updateMany(  
|   {gender: 'm'},  
|   {$inc: {vampires: 5}}  
| )  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 8,  
  modifiedCount: 8,  
  upsertedCount: 0  
}  
learn> db.unicorns.find(  
|   {gender: 'm'},  
|   {_id: 0, name: 1, gender: 1, vampires: 1}  
| )  
[  
  { name: 'Horny', gender: 'm', vampires: 68 },  
  { name: 'Unicrom', gender: 'm', vampires: 187 },  
  { name: 'Rooooooodles', gender: 'm', vampires: 104 },  
  { name: 'Kenny', gender: 'm', vampires: 44 },  
  { name: 'Raleigh', gender: 'm', vampires: 7 },  
  { name: 'Pilot', gender: 'm', vampires: 59 },  
  { name: 'Dunx', gender: 'm', vampires: 170 },  
  { name: 'Barney', gender: 'm', vampires: 5 }  
]  
learn>
```

Рис. 39: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$inc` для увеличения числового значения поля `vampires`.

Выполнение практического задания 3.3.5

1. *Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.*
2. *Проверить содержимое коллекции `towns`.*

```
learn> db.towns.updateOne(
|   {name: 'Portland'},
|   {$unset: {'mayor.party': 1}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a186987bc2655abca876124'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> 
```

Рис. 40: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$unset` для удаления поля `party` из вложенного объекта `mayor`.

Выполнение практического задания 3.3.6

1. *Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.*
2. *Проверить содержимое коллекции unicorns.*

```

learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {$push: {loves: 'chocolate'}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a1863bdbbc2655abca87611f'),
    name: 'Pilot',
    loves: [
      'apple',
      'watermelon',
      'chocolate',
      'chocolate',
      'chocolate',
      'chocolate'
    ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn>

```

Рис. 41: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$push` для добавления нового элемента в массив предпочтений.

Выполнение практического задания 3.3.7

1. *Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.*
2. *Проверить содержимое коллекции unicorns.*

```

learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a186395bc2655abca876117'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>

```

Рис. 42: Изменение документа и проверка содержимого

В ходе выполнения задания был использован оператор `$addToSet` совместно с `$each` для добавления нескольких уникальных элементов в массив.

5.7 Удаление данных

Выполнение практического задания 3.4.1

- 1) Удаление городов с беспартийными мэрами

```

learn> db.towns.deleteMany({
|   "mayor.party": {$exists: false}
| })
{ acknowledged: true, deletedCount: 1 }
learn>

```

Рис. 43: Удаление данных и проверка содержимого

- 2) Очистка коллекции towns

```

learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()

learn>

```

Рис. 44: Очистка коллекции и проверка содержимого

- 3) Просмотр коллекций

```
learn> show collections
towns
unicorns
learn> 
```

Рис. 45: Получение списка коллекций

В ходе выполнения задания была выполнена полная очистка коллекции **towns** и просмотр списка существующих коллекций базы данных.

5.8 Ссылки в MongoDB

Выполнение практического задания 4.1.1

- 1) *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*
- 2) *Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.*
- 3) *Проверьте содержание коллекции единорогов.*

1) Создание коллекции habitats

```
learn> db.habitats.insertOne({
|   _id: "forest",
|   name: "Magic Forest",
|   description: "Magical forest where unicorns live"
| })
{ acknowledged: true, insertedId: 'forest' }
learn> db.habitats.insertOne({
|   _id: "mountains",
|   name: "Crystal Mountains",
|   description: "High mountain area with crystals"
| })
{ acknowledged: true, insertedId: 'mountains' }
learn> db.habitats.insertOne({
|   _id: "lake",
|   name: "Silver Lake",
|   description: "Quiet lake area with clean water"
| })
{ acknowledged: true, insertedId: 'lake' }
learn> 
```

Рис. 46: Создание коллекции habitats

В ходе выполнения задания была создана коллекция **habitats**, содержащая информацию о зонах обитания единорогов.

2) Добавление ссылок

```
learn> db.unicorns.updateOne(
|   {name: "Aurora"},
|   {$set: {habitat: {$ref: "habitats", $id: "forest"}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.updateOne(
|   {name: "Horny"},
|   {$set: {habitat: {$ref: "habitats", $id: "mountains"}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.updateOne(
|   {name: "Nimue"},
|   {$set: {habitat: {$ref: "habitats", $id: "lake"}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.updateOne(
|   {name: "Pilot"},
|   {$set: {habitat: {$ref: "habitats", $id: "forest"}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
| }
```

Рис. 47: Добавление ссылок для двух единорогов

3) Проверка содержимого коллекции

```
learn> db.unicorns.find(
|   {},
|   {_id: 0, name: 1, habitat: 1}
| )
|
| [
|   { name: 'Horny', habitat: DBRef('habitats', 'mountains') },
|   { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
|   { name: 'Unicrom' },
|   { name: 'Rooooooodles' },
|   { name: 'Solnara' },
|   { name: 'Ayna' },
|   { name: 'Kenny' },
|   { name: 'Raleigh' },
|   { name: 'Leia' },
|   { name: 'Pilot', habitat: DBRef('habitats', 'forest') },
|   { name: 'Nimue', habitat: DBRef('habitats', 'lake') },
|   { name: 'Dunx' },
|   { name: 'Barney' }
| ]
learn>
```

Рис. 48: Проверка содержимого коллекции unicorns

В ходе выполнения задания были созданы ссылки между коллекциями с использованием механизма DBRef.

5.9 Настройка индексов

Выполнение практического задания 4.2.1

1. Проверьте, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique.

```
learn> db.unicorns.createIndex(
|   {name: 1},
|   {unique: true}
| )
name_1
learn>
```

Рис. 49: Успешное создание индекса

Сейчас все имена уникальны:

- Horny
- Aurora
- Unicrom
- Barney
- Pilot
- и т.д.

Поэтому индекс успешно создан.

5.9 Управление индексами

Выполнение практического задания 4.3.1

- 1) Получите информацию о всех индексах коллекции `unicorns`.
- 2) Удалите все индексы, кроме индекса для идентификатора.
- 3) Попробуйте удалить индекс для идентификатора.

1) Получение информации обо всех индексах

```
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn>
```

Рис. 50: Получение списка индексов

2) Удаление всех индексов, кроме `_id`

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn>
```

Рис. 51: Удаление индексов и получение списка индексов

В ходе выполнения задания были удалены все пользовательские индексы коллекции `unicorns`. После удаления в коллекции остался только системный индекс `_id`.

3) Попытка удалить индекс `_id`

```
{ v: 2, key: { _id: 1 }, name: '_id_' }
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

Рис. 52: Получение ошибки при попытке удалить системный индекс `_id`

5.9 План запроса

Выполнение практического задания 4.4.1

- 1) Создайте объемную коллекцию `numbers`, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```
- 2) Выберите последних четыре документа.
- 3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)
- 4) Создайте индекс для ключа `value`.

- 5) Получите информацию о всех индексах коллекции *numbers*.
- 6) Выполните запрос 2.
- 7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
- 8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

1) Создание объемной коллекции *numbers*

```
mongoose: Error [InvalidOptions]: cannot drop _id index
learn> for(i = 0; i < 100000; i++) {
|   db.numbers.insertOne({value: i})
| }
| {
|   acknowledged: true,
|   insertedId: ObjectId('6a187037bc2655abca88e7c8')
| }
learn>
```

Рис. 53: Создание коллекции *numbers*

2) Выборка последних четырех документов

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a187037bc2655abca88e7c8'), value: 99999 },
  { _id: ObjectId('6a187037bc2655abca88e7c7'), value: 99998 },
  { _id: ObjectId('6a187037bc2655abca88e7c6'), value: 99997 },
  { _id: ObjectId('6a187037bc2655abca88e7c5'), value: 99996 }
]
learn>
```

Рис. 54: Получение последних 4 документов

В ходе выполнения задания был выполнен запрос для получения последних четырех документов коллекции *numbers*.

3) Анализ плана выполнения запроса без индекса

```

learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a187037bc2655abca88e7c8'), value: 99999 },
  { _id: ObjectId('6a187037bc2655abca88e7c7'), value: 99998 },
  { _id: ObjectId('6a187037bc2655abca88e7c6'), value: 99997 },
  { _id: ObjectId('6a187037bc2655abca88e7c5'), value: 99996 }
]
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 45,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,

```

Рис. 55: Получение анализа запроса без индекса

В поле `executionTimeMillis` значение 45, значит время выполнения запроса составило: 45ms.

4) Создание индекса по полю `value`

```

learn> db.numbers.createIndex({value: 1})
value_1
learn>

```

Рис. 56: Создание индекса по полю `value`

5) Получение информации об индексах

```

learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn>

```

Рис. 57: Получение списка индексов

6) Повторное выполнение запроса

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a187037bc2655abca88e7c8'), value: 99999 },
  { _id: ObjectId('6a187037bc2655abca88e7c7'), value: 99998 },
  { _id: ObjectId('6a187037bc2655abca88e7c6'), value: 99997 },
  { _id: ObjectId('6a187037bc2655abca88e7c5'), value: 99996 }
]
```

Рис. 58: Повторное выполнение запроса

7) Анализ плана запроса с индексом

```
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 1,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
```

Рис. 59: Получение анализа запроса с индексом

Теперь в поле `executionTimeMillis` значение 1, значит время выполнения запроса составило: 1ms за счет работы индекса.

Более эффективным является запрос с использованием индекса,

так как MongoDB выполняет поиск по индексированной структуре данных без полного просмотра всей коллекции.

Вывод

В ходе лабораторной работы были изучены основные операции работы с MongoDB. Была создана база данных learn и несколько коллекций, в которых выполнялись операции добавления, выборки, изменения и удаления документов.

Были рассмотрены запросы с условиями, сортировкой, ограничением количества результатов и проекцией полей. Также были использованы логические операторы, работа с массивами, вложенными объектами и курсорами.

Дополнительно были выполнены агрегированные запросы, созданы ссылки между коллекциями, изучена работа с индексами и проанализирован план выполнения запроса с помощью explain("executionStats").

Таким образом, в результате работы были получены практические навыки выполнения CRUD-операций, агрегации, настройки индексов и анализа запросов в MongoDB.